

Threat Analysis and Detection Engineering Handbook

Windows, Linux, SPL, and KQL Edition

Prepared for Leonard

2026

Contents

0.1	Introduction	3
0.2	Splunk and SPL Fundamentals	3
0.3	Threat Hunting Fundamentals	5
0.4	Detection Engineering Fundamentals	6
0.5	Dashboard Creation	8
0.6	Linux Investigation Commands	10
0.7	CTI Workflows	11
0.8	Threat Hunting Playbooks	13
0.9	Detection Playbooks	16
0.10	Advanced Detection Patterns	20

0.1 Introduction

Security operations rely on the ability to interpret activity across systems, networks, and users. This handbook is designed as a practical, long-term reference for CTI analysts, threat hunters, and detection engineers who work with Splunk, Microsoft Sentinel, Linux systems, and enterprise telemetry. The focus is on real-world techniques, not abstract theory—everything included here is meant to support investigations, detection logic, and day-to-day analytical work.

Splunk and its query language, SPL, remain central tools for log analysis and threat hunting. They allow analysts to explore telemetry, identify suspicious behavior, and build detections with precision. Because many organizations also rely on Microsoft Sentinel, each SPL example in this handbook is paired with an equivalent KQL version. This dual approach ensures that the concepts and workflows you learn are transferable across platforms, strengthening your versatility as an analyst.

Threat hunting and CTI work require a strong understanding of how systems behave under normal conditions and how adversaries attempt to evade detection. To support this, the handbook includes Linux command references, process inspection techniques, and network investigation commands that are essential during hunts and incident response. These commands complement the SPL and KQL examples, giving you both endpoint-level and SIEM-level visibility.

Throughout the sections that follow, you will find structured examples, detection patterns, dashboard concepts, and practical workflows that reflect real-world analyst tasks. Whether you are building detections, performing hunts, analyzing malware behavior, or designing dashboards, this handbook is intended to serve as a reliable companion as you advance your career in cyber threat intelligence and security operations.

0.2 Splunk and SPL Fundamentals

Splunk is one of the most widely used platforms for log analysis, threat hunting, and detection engineering. Its strength lies in its ability to ingest large volumes of telemetry from diverse sources and make that data searchable in near real time. For a CTI analyst or threat hunter, Splunk becomes the primary interface for understanding system behavior, identifying anomalies, and validating hypotheses. This section introduces the core concepts of Splunk and its query language, SPL, in a way that supports long-term analytical work.

At its core, SPL (Search Processing Language) is designed to filter, transform, and analyze event data. Most SPL searches follow a simple pattern: start with an index, apply filters, and then use commands such as **stats**, **table**, or **eval** to shape the results. The goal is not to memorize every command, but to understand how to combine them to answer investigative questions. Because many organizations also rely on Microsoft Sentinel, each SPL example here is paired with an equivalent KQL version to reinforce cross-platform skills.

Basic Search Structure

Before diving into threat hunting examples, it is useful to understand the basic structure of an SPL search. The following example retrieves Windows process creation events from Sysmon:

SPL Example

```
index=sysmon EventCode=1
| table _time, Computer, User, Image, CommandLine
```

KQL Equivalent

```
Sysmon
| where EventID == 1
| project TimeGenerated, Computer, User, Image, CommandLine
```

This simple pattern—filter, then shape—is the foundation of nearly every SPL and KQL query you will write.

Filtering and Field Matching

Filtering is one of the most important skills in SPL. Analysts often begin with broad searches and gradually narrow them down as they learn more about the activity. The following example searches for PowerShell executions that include encoded commands, a common sign of obfuscation:

SPL Example

```
index=wineventlog EventCode=4104
| search CommandLine="*EncodedCommand"
```

KQL Equivalent

```
PowerShellEvent
| where EventID == 4104
| where CommandLine contains "EncodedCommand"
```

These types of queries are frequently used in threat hunting, detection engineering, and CTI validation.

Aggregation and Pattern Analysis

Threat hunters often need to identify unusual patterns rather than single events. Aggregation commands such as **stats** allow analysts to group events and look for anomalies. The following example identifies accounts with repeated failed logons:

SPL Example

```
index=wineventlog EventCode=4625
| stats count by AccountName, WorkstationName
| where count > 10
```

KQL Equivalent

```
SecurityEvent
| where EventID == 4625
| summarize Count=count() by Account, Workstation
| where Count > 10
```

This type of aggregation is foundational for detecting brute-force attempts, credential misuse, and lateral movement.

Process Relationships

Understanding parent-child process relationships is essential for detecting suspicious execution chains. SPL makes this easy by correlating fields such as **ParentImage** and **Image**. The following example highlights unusual parent-child pairs:

SPL Example

```
index=sysmon EventCode=1
| stats count by ParentImage, Image
| where count < 5
```

KQL Equivalent

```
Sysmon
| where EventID == 1
| summarize Count=count() by ParentImage, Image
| where Count < 5
```

This is a common technique in threat hunting to identify rare or suspicious execution paths.

Why These Fundamentals Matter

These examples form the foundation of nearly all detection and hunting work. Whether you are validating CTI findings, building dashboards, or investigating an alert, the ability to quickly filter, aggregate, and correlate telemetry is essential. As the handbook progresses, these fundamentals will be expanded into full detection patterns, hunting playbooks, and dashboard designs that reflect real-world analyst workflows.

0.3 Threat Hunting Fundamentals

Threat hunting is the proactive search for adversary activity that has evaded automated detection. Unlike alert triage, hunting begins without a predefined signal; instead, the analyst forms hypotheses based on telemetry, threat intelligence, and knowledge of system behavior. The goal is to identify subtle patterns, weak signals, and behavioral anomalies that indicate malicious activity. This section introduces the core mindset and techniques used in modern threat hunting, supported by SPL and KQL examples that reflect real investigative workflows.

A hunt typically begins with a hypothesis. This can be driven by CTI (“a threat actor is known to use encoded PowerShell commands”), by anomalies in telemetry (“a spike in failed logons”), or by environmental knowledge (“this server should never initiate outbound connections”). Once a hypothesis is defined, the hunter uses SPL or KQL to explore relevant telemetry, pivot across related events, and refine the search until meaningful patterns emerge. This iterative process is what separates hunting from simple querying.

One of the most common starting points in a hunt is process creation activity. Sysmon Event ID 1 provides detailed insight into how processes are launched, including parent-child relationships and command-line arguments. The following example searches for suspicious PowerShell usage, a frequent indicator of malicious activity:

SPL Example

```
index=sysmon EventCode=1 Image="*powershell.exe"
| search CommandLine="*EncodedCommand*"
| table _time, Computer, User, ParentImage, Image, CommandLine
```

KQL Equivalent

```
Sysmon
| where EventID == 1 and Image endswith "powershell.exe"
| where CommandLine contains "EncodedCommand"
| project TimeGenerated, Computer, User, ParentImage, Image, CommandLine
```

Another common hunting technique is identifying rare or unusual behavior. Adversaries often stand out not because their activity is obviously malicious, but because it is uncommon compared to normal system patterns. The following example highlights rare parent-child process pairs:

SPL Example

```
index=sysmon EventCode=1
| stats count by ParentImage, Image
| where count < 3
```

KQL Equivalent

```
Sysmon
| where EventID == 1
| summarize Count=count() by ParentImage, Image
| where Count < 3
```

Authentication activity is another rich source of hunting leads. Repeated failed logons, unusual logon types, or logons from unexpected hosts can indicate credential misuse or lateral movement. The following example identifies accounts with excessive failed logons:

SPL Example

```
index=wineventlog EventCode=4625
| stats count by AccountName, WorkstationName
| where count > 10
```

KQL Equivalent

```
SecurityEvent
| where EventID == 4625
| summarize Count=count() by Account, Workstation
| where Count > 10
```

Network telemetry also plays a critical role in hunting. DNS logs, proxy logs, and NetFlow data can reveal command-and-control activity, data exfiltration, or lateral movement. The following example identifies rare outbound connections:

SPL Example

```
index=sysmon EventCode=3
| stats count by DestinationIp, DestinationPort
| where count < 5
```

KQL Equivalent

```
Sysmon
| where EventID == 3
| summarize Count=count() by DestinationIp, DestinationPort
| where Count < 5
```

Threat hunting is ultimately about pattern recognition and curiosity. The examples in this section demonstrate the foundational techniques—filtering, aggregation, rarity analysis, and behavioral correlation—that underpin more advanced hunts. As the handbook progresses, these fundamentals will be expanded into full hunting playbooks and detection patterns that reflect real-world adversary behavior.

0.4 Detection Engineering Fundamentals

Detection engineering focuses on identifying the behaviors, patterns, and signals that indicate malicious activity within an environment. While threat hunting is exploratory and hypothesis-driven, detection engineering is structured and repeatable. The goal is to translate knowledge of adversary behavior into reliable,

high-fidelity detections that operate continuously in a SIEM such as Splunk or Microsoft Sentinel. This section introduces the foundational concepts and provides practical SPL and KQL examples that reflect real-world detection workflows.

A good detection begins with understanding the behavior you want to identify. Instead of relying on static indicators such as IP addresses or file hashes, modern detections focus on techniques—how an adversary executes code, moves laterally, steals credentials, or evades defenses. These behaviors tend to be more stable over time and more resilient to simple changes in tooling. For example, while a malware family may change its command-and-control infrastructure frequently, its use of encoded PowerShell commands or suspicious parent-child process chains often remains consistent.

One of the most common detection patterns involves monitoring for suspicious command-line activity. PowerShell is frequently abused by adversaries because it is built into Windows and provides extensive system access. The following example detects encoded PowerShell commands, which are often used to hide malicious intent:

SPL Example

```
index=sysmon EventCode=1 Image="*powershell.exe"
| search CommandLine="*EncodedCommand*"
| table _time, Computer, User, ParentImage, Image, CommandLine
```

KQL Equivalent

```
Sysmon
| where EventID == 1 and Image endswith "powershell.exe"
| where CommandLine contains "EncodedCommand"
| project TimeGenerated, Computer, User, ParentImage, Image, CommandLine
```

Another foundational detection pattern focuses on parent-child process anomalies. Legitimate processes tend to follow predictable execution chains, while malicious activity often introduces unusual or rare combinations. The following example identifies uncommon parent-child pairs:

SPL Example

```
index=sysmon EventCode=1
| stats count by ParentImage, Image
| where count < 3
```

KQL Equivalent

```
Sysmon
| where EventID == 1
| summarize Count=count() by ParentImage, Image
| where Count < 3
```

Authentication-based detections are equally important. Excessive failed logons, unusual logon types, or logons from unexpected hosts can indicate credential misuse or lateral movement. The following example identifies accounts with repeated failed logons:

SPL Example

```
index=wineventlog EventCode=4625
| stats count by AccountName, WorkstationName
| where count > 10
```

KQL Equivalent

```
SecurityEvent
| where EventID == 4625
| summarize Count=count() by Account, Workstation
| where Count > 10
```

Network-based detections help identify command-and-control activity, data exfiltration, or lateral movement. Rare outbound connections are often a strong signal of malicious behavior:

SPL Example

```
index=sysmon EventCode=3
| stats count by DestinationIp, DestinationPort
| where count < 5
```

KQL Equivalent

```
Sysmon
| where EventID == 3
| summarize Count=count() by DestinationIp, DestinationPort
| where Count < 5
```

Effective detection engineering also requires tuning and validation. A detection should be tested against real telemetry to ensure it produces meaningful results without overwhelming analysts with noise. This often involves adding filters, whitelisting known-good activity, or enriching events with additional context. Over time, detections should be reviewed and updated as adversary behavior evolves or as new telemetry sources become available.

The examples in this section represent the core building blocks of detection engineering: identifying suspicious behavior, correlating related events, and transforming threat intelligence into actionable, automated logic. These fundamentals will be expanded in later sections into full detection patterns, dashboards, and playbooks that reflect real-world adversary techniques.

0.5 Dashboard Creation

Dashboards transform raw telemetry into visual insights that help analysts monitor activity, validate detections, and identify anomalies at scale. For a CTI analyst or threat hunter, dashboards serve two purposes: they provide situational awareness, and they act as investigative starting points. A well-designed dashboard highlights deviations from normal behavior, exposes suspicious trends, and allows analysts to pivot quickly into deeper searches. This section introduces the core principles of dashboard design and provides practical SPL and KQL examples that can be used to build effective visualizations in Splunk and Microsoft Sentinel.

A good dashboard begins with a clear objective. Instead of trying to visualize everything, focus on the behaviors that matter most: process execution, authentication patterns, network activity, and high-risk events. Each panel should answer a specific question, such as “Which processes are most active?”, “Which accounts are failing logons?”, or “Which hosts are making rare outbound connections?” By structuring dashboards around investigative questions, you ensure that each visualization provides actionable insight rather than noise.

Process Activity Panel

Process creation is one of the most valuable signals for both hunting and detection engineering. A dashboard panel that highlights the most common processes helps establish a baseline and makes anomalies easier to spot.

SPL Example

```
index=sysmon EventCode=1
| stats count by Image
| sort - count
```

KQL Equivalent

```
Sysmon
| where EventID == 1
| summarize Count=count() by Image
| order by Count desc
```

This panel helps analysts quickly identify unusual binaries or unexpected spikes in activity.

Authentication Trends Panel

Authentication logs reveal credential misuse, brute-force attempts, and lateral movement. A dashboard panel that tracks failed logons over time provides early warning of suspicious behavior.

SPL Example

```
index=wineventlog EventCode=4625
| timechart count by AccountName
```

KQL Equivalent

```
SecurityEvent
| where EventID == 4625
| summarize Count=count() by bin(TimeGenerated, 1h), Account
| render timechart
```

This visualization helps identify targeted accounts or sudden increases in authentication failures.

Network Activity Panel

Network telemetry is essential for detecting command-and-control activity, lateral movement, and exfiltration. A dashboard panel that highlights rare outbound connections can reveal early signs of compromise.

SPL Example

```
index=sysmon EventCode=3
| stats count by DestinationIp, DestinationPort
| where count < 5
```

KQL Equivalent

```
Sysmon
| where EventID == 3
| summarize Count=count() by DestinationIp, DestinationPort
| where Count < 5
```

This panel surfaces low-frequency connections that may indicate malicious activity.

High-Risk Event Panel

Some events are inherently high-risk and deserve dedicated visibility. Examples include PowerShell execution, scheduled task creation, and privilege escalation attempts. A dashboard panel that highlights these events helps analysts quickly identify suspicious behavior.

SPL Example

```
index=sysmon (EventCode=1 Image="*powershell.exe") OR EventCode=13 OR EventCode=7
| stats count by EventCode, Image, User
```

KQL Equivalent

```
Sysmon
| where EventID in (1, 7, 13)
| summarize Count=count() by EventID, Image, User
```

This panel consolidates multiple high-risk signals into a single view.

Design Principles

Effective dashboards follow a few simple principles. Panels should be arranged from broad to specific, allowing analysts to move from general trends to detailed insights. Visualizations should be easy to interpret at a glance, with clear labels and minimal clutter. Most importantly, dashboards should be actionable: every panel should support a pivot into deeper investigation. Over time, dashboards should be refined based on analyst feedback and evolving threat behavior.

The examples in this section provide the foundation for building dashboards that support real-world CTI, hunting, and detection workflows. Later sections will expand on these concepts with full dashboard layouts and advanced visualizations tailored to specific adversary techniques.

0.6 Linux Investigation Commands

Linux systems are common targets in enterprise environments, and understanding how to investigate them is essential for CTI analysts, threat hunters, and detection engineers. While SIEM telemetry provides high-level visibility, direct Linux commands offer immediate insight into processes, users, network activity, and system integrity. This section provides a focused set of commands that support real-world investigations, along with explanations of when and why each command is useful.

Investigations often begin with understanding what is running on a system. The **ps** and **top** utilities reveal active processes, their parent-child relationships, and resource usage. These commands help identify suspicious binaries, unexpected services, or processes running under unusual accounts. For example, the following command lists all running processes with full details:

```
ps aux
```

To investigate a specific process, analysts often need to inspect its parent process, command-line arguments, and open files. The **lsuf** command is particularly valuable for identifying network connections or files associated with a suspicious process:

```
lsuf -p <PID>
```

Network visibility is equally important. Commands such as **ss** and **netstat** reveal active connections, listening ports, and associated processes. These commands help identify unauthorized services, reverse shells, or unexpected outbound traffic:

```
ss -tulnp
```

Authentication and privilege misuse are common indicators of compromise. Linux logs authentication events in `/var/log/auth.log`, which can be inspected directly or forwarded to Splunk or Sentinel. The following command shows recent authentication attempts:

```
grep "session" /var/log/auth.log
```

File integrity is another critical area. Attackers often modify system binaries, create persistence mechanisms, or drop malicious files. The `find` command helps locate recently modified files, which can reveal tampering or unauthorized activity:

```
find / -type f -mtime -1 2>/dev/null
```

System services and startup mechanisms are frequent targets for persistence. The `systemctl` command provides insight into active services, failed services, and recently modified units:

```
systemctl list-units --type=service
```

For deeper forensic analysis, `auditd` provides detailed system call logging, including file access, privilege escalation attempts, and execution events. When enabled, audit logs can be queried directly:

```
ausearch -m EXECVE
```

Linux investigations often require correlating endpoint findings with SIEM telemetry. For example, a suspicious process discovered with `ps aux` can be cross-referenced with Sysmon for Linux or audit logs ingested into Splunk or Sentinel. The following SPL and KQL examples illustrate how to pivot from endpoint commands to SIEM queries:

SPL Example

```
index=linux_logs process_name="bash"
| table _time, host, user, parent_process, command
```

KQL Equivalent

```
LinuxSysmon
| where ProcessName == "bash"
| project TimeGenerated, HostName, User, ParentProcessName, CommandLine
```

These combined techniques—endpoint commands plus SIEM queries—form the foundation of Linux threat hunting and incident response. As the handbook progresses, these commands will be integrated into full hunting playbooks and detection patterns tailored to Linux environments.

0.7 CTI Workflows

Cyber Threat Intelligence becomes most valuable when it directly supports operational teams. Effective CTI workflows bridge the gap between intelligence reporting and hands-on defensive work by transforming threat insights into practical actions. For a CTI analyst, threat hunter, or detection engineer, the goal is not simply to collect intelligence but to operationalize it—turning adversary behaviors into hypotheses, detections, dashboards, and investigative leads. This section outlines the core workflows that enable CTI to drive meaningful security outcomes.

A typical CTI workflow begins with the identification of relevant intelligence. This may come from threat reports, malware analysis, internal telemetry, or external feeds. The analyst evaluates the intelligence to

determine what behaviors, techniques, or infrastructure are most relevant to the organization. Instead of focusing on short-lived indicators such as IP addresses or domains, the emphasis is placed on techniques—how the adversary gains access, executes code, moves laterally, or evades detection. These behaviors form the foundation for operational work.

Once relevant behaviors are identified, the next step is translation. CTI analysts convert high-level intelligence into concrete questions that can be answered with telemetry. For example, if a threat actor is known to use encoded PowerShell commands, the operational question becomes: “Where in our environment do we see PowerShell with encoded commands?” This translation step is critical because it connects intelligence to the data sources available in Splunk or Sentinel.

The translated intelligence then becomes the basis for threat hunting. Analysts use SPL or KQL to search for behaviors associated with the threat actor. The following example illustrates how CTI-driven intelligence about encoded PowerShell commands becomes a hunt:

SPL Example

```
index=sysmon EventCode=1 Image="*powershell.exe"
| search CommandLine="*EncodedCommand*"
| table _time, Computer, User, ParentImage, Image, CommandLine
```

KQL Equivalent

```
Sysmon
| where EventID == 1 and Image endswith "powershell.exe"
| where CommandLine contains "EncodedCommand"
| project TimeGenerated, Computer, User, ParentImage, Image, CommandLine
```

If the hunt reveals suspicious activity, the workflow moves into detection engineering. The same behavioral pattern is refined into a repeatable detection that continuously monitors for similar activity. This ensures that the organization is protected not only against the current threat but also against future variants that use the same technique.

CTI workflows also support dashboard creation. Intelligence about adversary behavior informs which panels should be included in dashboards and how they should be structured. For example, if a threat actor frequently uses scheduled tasks for persistence, a dashboard panel that highlights scheduled task creation becomes valuable. CTI ensures that dashboards remain aligned with real-world threats rather than generic system activity.

Another key component of CTI workflows is enrichment. When analysts encounter suspicious events in Splunk or Sentinel, CTI provides context that helps determine whether the activity is benign or malicious. For example, a suspicious domain discovered in DNS logs can be enriched with intelligence about whether it is associated with a known threat actor. This enrichment accelerates investigations and improves decision-making.

Finally, CTI workflows rely on feedback loops. Threat hunters, SOC analysts, and detection engineers provide insight into which intelligence is useful and which detections generate noise. This feedback helps refine intelligence requirements and ensures that CTI remains aligned with operational needs. Over time, this iterative process strengthens the organization’s defensive posture and improves the quality of both intelligence and detections.

The workflows described in this section form the backbone of operational CTI. By connecting intelligence to telemetry, hunts, detections, dashboards, and investigations, CTI becomes a practical force multiplier rather than a theoretical exercise. These workflows will be expanded in later sections into full playbooks and detection patterns tailored to specific adversary techniques.

0.8 Threat Hunting Playbooks

Threat hunting playbooks provide structured, repeatable approaches for investigating specific adversary behaviors. Unlike detections, which run continuously, playbooks guide analysts through deeper manual exploration. Each playbook begins with a hypothesis, identifies relevant telemetry, and provides SPL, KQL, and Linux commands that support the investigation. These playbooks are designed to be practical, adaptable, and aligned with real-world adversary techniques.

Playbook: Suspicious PowerShell Activity

Hypothesis: An adversary is using PowerShell to execute encoded or obfuscated commands.

Relevant Telemetry: Sysmon Event ID 1, PowerShell logs (4103, 4104), process command-line arguments.

SPL Example

```
index=sysmon EventCode=1 Image="*powershell.exe"
| search CommandLine="*EncodedCommand*"
| table _time, Computer, User, ParentImage, Image, CommandLine
```

KQL Equivalent

```
Sysmon
| where EventID == 1 and Image endswith "powershell.exe"
| where CommandLine contains "EncodedCommand"
| project TimeGenerated, Computer, User, ParentImage, Image, CommandLine
```

Linux Pivot (Sysmon for Linux)

```
grep -i "powershell" /var/log/sysmon/sysmon.log
```

This playbook helps identify obfuscated execution, a common technique in malware delivery and post-exploitation.

Playbook: Credential Access Attempts

Hypothesis: An adversary is attempting to brute-force or misuse credentials.

Relevant Telemetry: Windows Event ID 4625, 4624, Linux auth logs, network logs.

SPL Example

```
index=wineventlog EventCode=4625
| stats count by AccountName, WorkstationName
| where count > 10
```

KQL Equivalent

```
SecurityEvent
| where EventID == 4625
| summarize Count=count() by Account, Workstation
| where Count > 10
```

Linux Pivot

```
grep "Failed_password" /var/log/auth.log | awk '{print $9}' | sort | uniq -c
```

This playbook identifies brute-force attempts, password spraying, and compromised accounts.

Playbook: Lateral Movement via Remote Execution

Hypothesis: An adversary is moving laterally using remote execution tools such as PsExec, WMI, or SSH.

Relevant Telemetry: Sysmon Event ID 1, 3, Windows Event ID 7045, Linux SSH logs.

SPL Example (PsExec)

```
index=sysmon EventCode=1 Image="*psexec*"
| table _time, Computer, User, ParentImage, Image, CommandLine
```

KQL Equivalent

```
Sysmon
| where EventID == 1 and Image contains "psexec"
| project TimeGenerated, Computer, User, ParentImage, Image, CommandLine
```

Linux Pivot (SSH)

```
grep "Accepted_password" /var/log/auth.log
grep "Failed_password" /var/log/auth.log
```

This playbook helps identify unauthorized remote execution and lateral movement.

Playbook: Rare Network Connections

Hypothesis: A compromised host is communicating with an unusual or suspicious external destination.

Relevant Telemetry: Sysmon Event ID 3, firewall logs, DNS logs.

SPL Example

```
index=sysmon EventCode=3
| stats count by DestinationIp, DestinationPort
| where count < 5
```

KQL Equivalent

```
Sysmon
| where EventID == 3
| summarize Count=count() by DestinationIp, DestinationPort
| where Count < 5
```

Linux Pivot

```
ss -tulnp
netstat -antp
```

This playbook is effective for identifying command-and-control channels and exfiltration paths.

Playbook: Persistence Mechanisms

Hypothesis: An adversary has established persistence on a host.

Relevant Telemetry: Scheduled tasks, services, registry modifications, Linux cron jobs.

SPL Example (Scheduled Tasks)

```
index=wineventlog EventCode=4698
| table _time, TaskName, Author, Command
```

KQL Equivalent

```
SecurityEvent
| where EventID == 4698
| project TimeGenerated, TaskName, Author, Command
```

Linux Pivot (Cron Jobs)

```
cat /etc/crontab
ls -al /etc/cron.d/
```

This playbook helps identify long-term footholds established by attackers.

Playbook: Suspicious File Modifications

Hypothesis: An adversary is modifying system files or dropping malicious payloads.

Relevant Telemetry: Sysmon Event ID 11, Linux file modification logs.

SPL Example

```
index=sysmon EventCode=11
| table _time, Computer, User, TargetFilename
```

KQL Equivalent

```
Sysmon
| where EventID == 11
| project TimeGenerated, Computer, User, TargetFilename
```

Linux Pivot

```
find / -type f -mtime -1 2>/dev/null
```

This playbook is useful for detecting tampering, malware staging, and unauthorized changes.

Playbook Structure Summary

Each playbook follows the same structure:

- Hypothesis
- Relevant telemetry
- SPL query
- KQL equivalent
- Linux investigation commands

This consistent format ensures that hunts are repeatable, scalable, and aligned with real-world adversary behavior.

0.9 Detection Playbooks

Detection playbooks translate knowledge of adversary behavior into concrete, repeatable logic that can run continuously in Splunk or Microsoft Sentinel. Unlike hunting playbooks, which are exploratory and analyst-driven, detection playbooks are designed for automation: they define what to look for, how to detect it, how to investigate it, and how to tune it over time. For a CTI analyst or detection engineer, these playbooks form the backbone of a mature detection program and serve as a long-term reference for building and maintaining high-fidelity alerts.

Each detection playbook in this section follows a consistent structure. It begins with a short description of the behavior and why it matters, followed by the required telemetry. Then it provides SPL and KQL detection logic, along with Linux investigation commands where relevant. Finally, it discusses tuning considerations and common sources of false positives. The goal is not only to provide queries, but to explain how and why they work, so that they can be adapted to different environments and threat landscapes.

Detection Playbook: Encoded PowerShell Commands

Behavior and Rationale

Adversaries frequently abuse PowerShell to execute malicious code, download payloads, or perform reconnaissance. To evade simple string-based detections, they often use the `-EncodedCommand` parameter, which allows them to supply Base64-encoded commands. This behavior is highly suspicious in most environments and is a strong candidate for a high-priority detection, especially on servers and high-value endpoints.

Required Telemetry

This detection relies on detailed process creation logs that include command-line arguments. On Windows, Sysmon Event ID 1 or Windows Event ID 4688 with full command-line logging is ideal. In Splunk, these events are typically found in indexes such as `sysmon` or `wineventlog`. In Sentinel, the equivalent data is often in `Sysmon` or `SecurityEvent` tables.

SPL Detection Logic

```
index=sysmon EventCode=1 Image="*powershell.exe"
| search CommandLine="*EncodedCommand*"
| stats count by Computer, User, ParentImage, Image, CommandLine
| where count >= 1
```

KQL Detection Logic

```
Sysmon
| where EventID == 1 and Image endswith "powershell.exe"
| where CommandLine contains "EncodedCommand"
| summarize Count=count() by Computer, User, ParentImage, Image, CommandLine
| where Count >= 1
```

This logic looks for any PowerShell process that includes the `EncodedCommand` parameter. In many environments, even a single occurrence is worth investigating. The aggregation step (`stats` / `summarize`) helps group similar events and makes it easier to review repeated activity.

Linux Investigation Pivot

On systems with Sysmon for Linux or similar logging, analysts can pivot from SIEM detections to local logs:

```
grep -i "powershell" /var/log/sysmon/sysmon.log | grep -i "EncodedCommand"
```


Tuning and False Positives

Some administrative scripts or management tools may legitimately use encoded PowerShell, especially in automation-heavy environments. To reduce noise, detections can be tuned by excluding known management accounts, specific hosts (such as configuration servers), or known-good scripts. Over time, a whitelist of expected encoded PowerShell usage can be maintained, while all other occurrences remain high-priority alerts.

Detection Playbook: Excessive Failed Logons

Behavior and Rationale

Repeated failed logons are a classic indicator of brute-force attacks, password spraying, or credential guessing. While individual failures are common and often benign, patterns of repeated failures against specific accounts or from specific hosts are highly suspicious. Detecting these patterns early helps prevent account compromise and lateral movement.

Required Telemetry

On Windows, failed logons are recorded as Event ID 4625 in the Security log. In Splunk, these events are typically found in `wineventlog` or similar indexes. In Sentinel, they appear in the `SecurityEvent` table. For Linux, failed SSH logons are recorded in `/var/log/auth.log`.

SPL Detection Logic

```
index=wineventlog EventCode=4625
| stats count by AccountName, WorkstationName
| where count > 10
```

KQL Detection Logic

```
SecurityEvent
| where EventID == 4625
| summarize Count=count() by Account, Workstation
| where Count > 10
```

This detection identifies accounts that have more than ten failed logons from a given workstation. The threshold can be adjusted based on the environment's normal behavior. In high-noise environments, it may be useful to add a time window (for example, failures within one hour) to focus on concentrated attack patterns.

Linux Investigation Pivot

On Linux systems, failed SSH logons can be reviewed directly:

```
grep "Failed password" /var/log/auth.log | awk '{print $9}' | sort | uniq -c
```

This command shows how many times each user has experienced failed logons, which can be correlated with SIEM detections.

Tuning and False Positives

False positives often arise from misconfigured services, automated scripts with outdated credentials, or users repeatedly entering incorrect passwords. Tuning should focus on excluding known noisy systems and service accounts, while maintaining strict thresholds for privileged accounts and high-value targets. Additional context, such as geolocation or device type, can further improve fidelity.

Detection Playbook: Rare Parent-Child Process Relationships

Behavior and Rationale

Legitimate processes tend to follow predictable parent-child relationships. For example, `explorer.exe` commonly spawns user applications, while `services.exe` spawns services. When an unusual parent process launches a sensitive child process (such as a scripting engine or system tool), it may indicate process injection, masquerading, or other malicious activity. Detecting rare parent-child combinations is a powerful behavioral technique.

Required Telemetry

This detection relies on detailed process creation logs with parent and child process information. Sysmon Event ID 1 is ideal, as it includes `ParentImage` and `Image` fields. In Splunk, these events are typically stored in a `sysmon` index; in Sentinel, they appear in the `Sysmon` table.

SPL Detection Logic

```
index=sysmon EventCode=1
| stats count by ParentImage, Image
| where count < 3
```

KQL Detection Logic

```
Sysmon
| where EventID == 1
| summarize Count=count() by ParentImage, Image
| where Count < 3
```

This detection highlights parent-child pairs that occur fewer than three times in the dataset. These rare combinations are strong candidates for further investigation. In practice, this detection is often run over a rolling time window (for example, the last seven days) to reflect current behavior.

Linux Investigation Pivots

On Linux, process relationships can be inspected using `ps` and `pstree`:

```
ps aux --forest
```

This view helps analysts confirm whether a suspicious process tree observed in SIEM logs is legitimate or malicious.

Tuning and False Positives

Rare does not always mean malicious. New software deployments, updates, or one-time administrative tasks can create unusual process chains. Tuning involves building a baseline of known-good rare pairs and excluding them from future alerts. Over time, the detection should focus on truly unexpected combinations, especially those involving scripting engines, interpreters, or administrative tools.

Detection Playbook: Rare Outbound Network Connections

Behavior and Rationale

Compromised hosts often communicate with external command-and-control servers or exfiltration endpoints that are not part of normal business operations. These connections are typically low-frequency and may target unusual ports or IP addresses. Detecting rare outbound connections helps identify such activity, especially when combined with CTI enrichment.

Required Telemetry

This detection relies on network connection logs that include destination IPs and ports. Sysmon Event ID 3 provides this information on Windows. In Splunk, these events are often stored in a `sysmon` or `network` index; in Sentinel, they appear in the `Sysmon` table or dedicated network tables.

SPL Detection Logic

```
index=sysmon EventCode=3
| stats count by DestinationIp, DestinationPort
| where count < 5
```

KQL Detection Logic

```
Sysmon
| where EventID == 3
| summarize Count=count() by DestinationIp, DestinationPort
| where Count < 5
```

This detection surfaces destination IP and port combinations that appear fewer than five times in the dataset. These rare connections can then be enriched with CTI (for example, checking against threat feeds) to determine whether they are associated with known malicious infrastructure.

Linux Investigation Pivot

On Linux, active connections can be inspected with:

```
ss -tulnp
```

This command shows listening and established connections, along with associated processes, which can be correlated with SIEM detections.

Tuning and False Positives

False positives may include legitimate but infrequent connections, such as one-off vendor support sessions or new cloud services. Tuning should focus on excluding known business destinations and internal infrastructure, while maintaining strict scrutiny for unknown external IPs, unusual ports, and connections from high-value systems.

Detection Playbook: Persistence via Scheduled Tasks and Cron Jobs

Behavior and Rationale

Persistence mechanisms allow adversaries to maintain access to a system even after reboots or user logouts. On Windows, scheduled tasks are a common persistence technique; on Linux, cron jobs serve a similar purpose. Detecting the creation or modification of these mechanisms is critical for identifying long-term footholds.

Required Telemetry

On Windows, scheduled task creation is recorded as Event ID 4698 in the Security log and may also appear in Task Scheduler logs. In Splunk, these events are typically found in `wineventlog`. In Sentinel, they appear in the `SecurityEvent` table. On Linux, cron configurations are stored in `/etc/crontab`, `/etc/cron.d/`, and user-specific cron files.

SPL Detection Logic

```
index=wineventlog EventCode=4698
| table _time, Computer, TaskName, Author, Command
```

KQL Detection Logic

```
SecurityEvent
| where EventID == 4698
| project TimeGenerated, Computer, TaskName, SubjectUserName, Command
```

This detection surfaces newly created scheduled tasks, including the task name, author, and command. In many environments, scheduled task creation is relatively rare and should be closely monitored, especially on servers and high-value endpoints.

Linux Investigation Pivot

On Linux, cron-based persistence can be reviewed with:

```
cat /etc/crontab
ls -al /etc/cron.d/
crontab -l
```

These commands reveal system-wide and user-specific cron jobs, which can be compared against known-good configurations.

Tuning and False Positives

Legitimate administrative tasks, backup jobs, and monitoring scripts often use scheduled tasks or cron jobs. Tuning should focus on identifying expected tasks (by name, author, or command) and excluding them from alerts. New or modified tasks that execute scripting engines, interpreters, or binaries from unusual paths should remain high-priority.

Summary

The detection playbooks in this section illustrate how to move from abstract knowledge of adversary behavior to concrete, automated logic in Splunk and Sentinel. Each playbook combines behavioral focus, clear telemetry requirements, SPL and KQL implementations, Linux investigation pivots, and practical tuning guidance. Over time, these playbooks can be expanded, refined, and adapted to new threats, forming a living detection library that supports your long-term growth as a CTI analyst, threat hunter, and detection engineer.

0.10 Advanced Detection Patterns

Advanced detection engineering focuses on identifying subtle, high-value behaviors that adversaries rely on to evade defenses, escalate privileges, and maintain persistence. These patterns go beyond simple indicators or basic anomalies; they capture the underlying techniques that remain consistent even as tooling, infrastructure, and malware families evolve. This section introduces several advanced detection patterns, each supported by SPL and KQL logic, along with Linux investigation pivots. The goal is to provide a deep understanding of how these behaviors manifest in telemetry and how to detect them reliably in enterprise environments.

Detection Pattern: Parent PID Spoofing

Behavior and Rationale

Parent PID spoofing occurs when an adversary manipulates the parent process ID of a newly created process to make it appear as though it was launched by a legitimate parent. This technique is used to evade behavioral detections that rely on parent-child relationships. While Windows does not natively allow arbitrary parent PID assignment, certain APIs and tools (such as process hollowing or direct system calls) can create misleading process trees. Detecting this behavior requires identifying inconsistencies between expected and observed parent-child relationships.

SPL Detection Logic

```

index=sysmon EventCode=1
| eval expected_parent=case(
    Image="*powershell.exe", "explorer.exe",
    Image="*cmd.exe", "explorer.exe",
    Image="*wscript.exe", "explorer.exe",
    1==1, "unknown")
| where NOT like(ParentImage, "%" . expected_parent . "%")
| table _time, Computer, User, ParentImage, Image, CommandLine

```

KQL Detection Logic

```

Sysmon
| where EventID == 1
| extend ExpectedParent = case(
    Image ends with "powershell.exe", "explorer.exe",
    Image ends with "cmd.exe", "explorer.exe",
    Image ends with "wscript.exe", "explorer.exe",
    "unknown")
| where ParentImage !contains ExpectedParent
| project TimeGenerated, Computer, User, ParentImage, Image, CommandLine

```

Linux Pivot

```
ps -eo pid,ppid,cmd --forest
```

Explanation

This detection identifies processes whose parent does not match expected norms. While not all mismatches are malicious, unexpected parent-child relationships are a strong indicator of process injection, hollowing, or spoofing.

Detection Pattern: LOLBins Abuse

Behavior and Rationale

Living-off-the-land binaries (LOLBins) are legitimate system tools that adversaries abuse to execute code, download payloads, or evade detection. Examples include `certutil`, `mshta`, `wmic`, and `rundll32`. Because these tools are signed and trusted, traditional signature-based detections often fail. Behavioral detections must focus on unusual usage patterns, command-line arguments, and execution context.

SPL Detection Logic

```

index=sysmon EventCode=1
| search Image IN ("*certutil.exe", "*mshta.exe", "*wmic.exe", "*rundll32.exe")
| table _time, Computer, User, ParentImage, Image, CommandLine

```

KQL Detection Logic

```

Sysmon
| where EventID == 1
| where Image ends with "certutil.exe"
    or Image ends with "mshta.exe"
    or Image ends with "wmic.exe"
    or Image ends with "rundll32.exe"
| project TimeGenerated, Computer, User, ParentImage, Image, CommandLine

```

Linux Pivot

```
grep -E "curl|wget|python|perl" /var/log/syslog
```

Explanation

This detection highlights the execution of high-risk binaries. Analysts should pay close attention to suspicious command-line arguments, such as Base64 strings, remote URLs, or script execution flags.

Detection Pattern: Credential Dumping Behavior

Behavior and Rationale

Credential dumping is a critical step in many intrusions, enabling adversaries to escalate privileges and move laterally. Tools such as Mimikatz, LSASS memory readers, and custom malware often interact with sensitive processes or registry hives. Detecting credential dumping requires monitoring for access to LSASS, suspicious handle requests, or unusual process memory reads.

SPL Detection Logic

```
index=sysmon EventCode=10 TargetImage="*lsass.exe"
| table _time, Computer, User, SourceImage, TargetImage, GrantedAccess
```

KQL Detection Logic

```
Sysmon
| where EventID == 10 and TargetImage endswith "lsass.exe"
| project TimeGenerated, Computer, User, SourceImage, TargetImage, GrantedAccess
```

Linux Pivot

```
grep -i "shadow" /var/log/auth.log
```

Explanation

Sysmon Event ID 10 captures process access attempts. Any process attempting to access LSASS with high privileges is suspicious and should be investigated immediately.

Detection Pattern: Registry-Based Persistence

Behavior and Rationale

Adversaries frequently use registry keys to establish persistence, especially via **Run**, **RunOnce**, and **Services**. Detecting modifications to these keys is essential for identifying long-term footholds.

SPL Detection Logic

```
index=sysmon EventCode=13
| search TargetObject="*\\Run*" OR TargetObject="*\\RunOnce*"
| table _time, Computer, User, TargetObject, Details
```

KQL Detection Logic

```
Sysmon
| where EventID == 13
| where TargetObject contains "\\Run" or TargetObject contains "\\RunOnce"
| project TimeGenerated, Computer, User, TargetObject, Details
```

Linux Pivot

```
ls -al ~/.config/autostart/
```

Explanation

Registry persistence is common in malware and should be monitored closely. Analysts should validate whether the modified entry corresponds to legitimate software.

Detection Pattern: Suspicious Service Creation**Behavior and Rationale**

Creating or modifying services is a powerful persistence and privilege escalation technique. Adversaries often create services that execute malicious binaries with SYSTEM privileges.

SPL Detection Logic

```
index=wineventlog EventCode=7045
| table _time, ServiceName, ImagePath, StartType, AccountName
```

KQL Detection Logic

```
SecurityEvent
| where EventID == 7045
| project TimeGenerated, ServiceName, ImagePath, StartType, Account
```

Linux Pivot

```
systemctl list-units --type=service
```

Explanation

Service creation is rare in most environments and should be treated as high-risk unless associated with known administrative activity.

Summary

The advanced detection patterns in this section focus on behaviors that adversaries rely on to evade defenses and maintain access. By understanding these patterns and implementing the associated SPL and KQL logic, analysts can build high-fidelity detections that remain effective even as threats evolve. These patterns form the foundation of a mature detection engineering program and support long-term operational resilience.